

ΚΕΦΑΛΑΙΟ 4

Συστοιχίες στη Fortran

Θεωρία, παραδείγματα και ασκήσεις εφαρμογής

Σκοπός. Να οργανώνεις πολλά δεδομένα του ίδιου τύπου σε μία συστοιχία, να τα προσπελαύνεις με δείκτες και να τα επεξεργάζεσαι με κατασκευαστές, RESHAPE, βρόχους και μορφοποιημένη εκτύπωση.

Βάση του εγχειριδίου: FORTRAN_2025, Περιγραφή Μαθήματος, Ασκήσεις_01 και Εργαστήριο 3 — Συστοιχίες 2025–2026. Η αρίθμηση 4.1–4.15 είναι η συμφωνημένη οργανωτική δομή μελέτης.

Χάρτης κεφαλαίου

Ενότητα	Κεντρική ιδέα
4.1–4.4	Έννοια, χαρακτηριστικά, 1D και 2D συστοιχίες
4.5–4.7	Κατασκευαστές, RESHAPE και σειρά τοποθέτησης
4.8–4.11	Πρόσβαση, DO/IF, πράξεις και εκτύπωση
4.12–4.15	Στρατηγική λύσης, συχνά λάθη, ασκήσεις, checklist

4.1 — Τι είναι συστοιχία

Μια συστοιχία είναι συλλογή δεδομένων του ίδιου τύπου με κοινό όνομα. Κάθε ξεχωριστό δεδομένο είναι στοιχείο της συστοιχίας και προσδιορίζεται από έναν ή περισσότερους ακέραιους δείκτες.

Όρος	Παράδειγμα με $a(6)$ και $x(2,3)$
όνομα	a, x
στοιχείο	$a(1), x(1,1)$
τάξη (rank)	1 για a , 2 για x
μέγεθος (size)	6 και $2 \times 3 = 6$
μορφή (shape)	$[6]$ και $[2,3]$

Κεντρική ιδέα: η δήλωση περιγράφει ολόκληρη τη συστοιχία· η χρήση με δείκτες αναφέρεται σε συγκεκριμένο στοιχείο.

4.2 — Βασικά χαρακτηριστικά: rank, size, shape και όρια

Η τάξη είναι το πλήθος διαστάσεων. Η έκταση κάθε διάστασης εμφανίζεται στη δήλωση. Σε αυτό το κεφάλαιο δουλεύουμε με στατικές συστοιχίες: η μορφή, άρα και το μέγεθος και η τάξη τους, είναι γνωστά από τη δήλωση και δεν αλλάζουν κατά την εκτέλεση.

```
integer :: v(5)
real :: m(2,3)
! v: rank 1, shape [5], size 5
! m: rank 2, shape [2,3], size 6
```

Με τη συνηθισμένη δήλωση, οι δείκτες ξεκινούν από το 1. Άρα επιτρέπονται $v(1)$ έως $v(5)$, και για το m οι τιμές $i=1..2, j=1..3$. Δείκτης έξω από τα όρια είναι σφάλμα.

4.3 — Μονοδιάστατες συστοιχίες (1D arrays)

Μια 1D συστοιχία ή διάνυσμα έχει έναν δείκτη. Η δήλωση και η εκχώρηση στοιχείων γίνονται ως εξής:

```
program one_d
implicit none
real :: a(5)
a(5) = 4.5
a(2) = 2.5
a(3) = 1.0
a(1) = a(5) + 2.0*a(3)
a(4) = a(5) - a(2)
print *, a
end program one_d
```

Γραμμή-γραμμή: δηλώνεται χώρος για 5 πραγματικούς αριθμούς. Δίνονται τιμές στα στοιχεία 5, 2 και 3. Μετά υπολογίζονται τα στοιχεία 1 και 4 από ήδη γνωστές τιμές. Η εκτύπωση εμφανίζει, με ελεύθερη μορφή, τις τιμές με σειρά δεικτών: 6.5, 2.5, 1.0, 2.0, 4.5.

4.4 — Δισδιάστατες συστοιχίες (2D arrays)

Μια 2D συστοιχία περιγράφεται με δύο εκτάσεις: πρώτη διάσταση και δεύτερη διάσταση. Συχνά τη διαβάζουμε ως γραμμές και στήλες, αλλά στη Fortran η σειρά εσωτερικής τοποθέτησης έχει ειδική σημασία (4.7).

```
real :: x(2,3)
x(1,1) = 2.5
x(2,3) = 9.0
```

Το $x(1,1)$ είναι ένα στοιχείο, όχι ολόκληρος πίνακας. Το $x(2,3)$ είναι το στοιχείο με πρώτο δείκτη 2 και δεύτερο δείκτη 3. Η τάξη είναι 2 και το μέγεθος είναι $2 \times 3 = 6$.

4.5 — Υπονοούμενες επαναλήψεις και array constructors

Ο array constructor δημιουργεί τιμές για μονοδιάστατη συστοιχία. Χρησιμοποιούμε τη σύγχρονη μορφή με αγκύλες.

```
integer :: i
real :: a(5)
a = [1.0, 3.0, 5.0, 7.0, 9.0]
a = [(i, i = 1, 5)]
```

Στην πρώτη εκχώρηση οι τιμές γράφονται μία-μία. Στη δεύτερη, η υπονοούμενη επανάληψη παράγει τις τιμές 1, 2, 3, 4, 5. Ο αριθμός των παραγόμενων στοιχείων πρέπει να ταιριάζει με το μέγεθος της συστοιχίας.

```
integer :: i
integer :: x(6)
x = [(2*i-1, i = 1, 6)]
print '(6(i4))', x
```

* Τι σημαίνει η εντολή εκτύπωσης: το print τυπώνει τις τιμές της συστοιχίας x. Το format (6(i4)) σημαίνει «τύπωσε 6 ακεραίους, καθέναν σε πεδίο πλάτους 4 χαρακτήρων». Έτσι οι τιμές μπαίνουν σε σωστά στοιχισμένες στήλες.

Τι εμφανίζει: 1 3 5 7 9 11. Για κάθε i υπολογίζεται ο περιττός αριθμός $2i-1$.

4.6 — RESHAPE

Η RESHAPE παίρνει μια ακολουθία στοιχείων και τη δίνει σε νέα μορφή. Χρησιμοποιείται εδώ για να μετατρέψουμε το 1D X σε 2D πίνακα.

```
integer :: x(6), y(2,3)
x = [1, 3, 5, 7, 9, 11]
y = reshape(x, [2,3])
```

Το [2,3] είναι η ζητούμενη μορφή: 2 στοιχεία στην πρώτη και 3 στη δεύτερη διάσταση. Χρειάζονται ακριβώς $2 \times 3 = 6$ στοιχεία· γι' αυτό η συστοιχία πηγής έχει μέγεθος 6.

Προσοχή: η μορφή [2,3] δεν σημαίνει αυτομάτως ότι η λίστα θα μπει πρώτα κατά γραμμές. Η προεπιλεγμένη συμπεριφορά της Fortran εξηγείται στην επόμενη ενότητα.

4.7 — Σειρά αποθήκευσης στις 2D συστοιχίες

Η RESHAPE δεν αλλάζει τις τιμές του X· αποφασίζει μόνο σε ποια θέση θα πάει η καθεμία. Για να μη χαθείς, σκέψου ότι παίρνουμε τα στοιχεία του X ένα-ένα, με τη σειρά 1, 3, 5, 7, 9, 11, και τα τοποθετούμε στις θέσεις του νέου πίνακα.

```
integer :: x(6), y(2,3), z(2,3)
x = [1, 3, 5, 7, 9, 11]
y = reshape(x, [2,3], order=[2,1]) ! κατά γραμμές
z = reshape(x, [2,3]) ! προεπιλογή: κατά στήλες
```

Περίπτωση A - χωρίς order (ο z). Η Fortran δίνει προτεραιότητα στον πρώτο δείκτη. Άρα, πριν αλλάξει ο δεύτερος δείκτης, αλλάζει ο πρώτος: $z(1,1)=1$, $z(2,1)=3$, $z(1,2)=5$, $z(2,2)=7$, $z(1,3)=9$, $z(2,3)=11$. Αν τον εκτυπώσουμε ανά γραμμή, βλέπουμε:

z: προεπιλογή	Στήλη 1	Στήλη 2	Στήλη 3
Γραμμή 1	1	5	9
Γραμμή 2	3	7	11

Περίπτωση B - με `order=[2,1]` (ο `y`). Τώρα δίνουμε προτεραιότητα στον δεύτερο δείκτη. Δηλαδή, πριν αλλάξει ο πρώτος δείκτης, αλλάζει ο δεύτερος: $y(1,1)=1$, $y(1,2)=3$, $y(1,3)=5$, $y(2,1)=7$, $y(2,2)=9$, $y(2,3)=11$. Άρα, ανά γραμμή:

<code>y, order=[2,1]</code>	Στήλη 1	Στήλη 2	Στήλη 3
Γραμμή 1	1	3	5
Γραμμή 2	7	9	11

Μικρός κανόνας: στην προεπιλογή το αριστερότερο `index` «τρέχει» γρηγορότερα. Στο `order=[2,1]` το δεξιότερο `index` «τρέχει» γρηγορότερα. Δεν αλλάζουν ούτε η μορφή `[2,3]` ούτε τα στοιχεία· αλλάζει μόνο η διαδρομή γεμίσματος.

Δεύτερο παράδειγμα. Με την ίδια ακολουθία αλλά μορφή `[3,2]`, η προεπιλογή δίνει γραμμές 1 7, 3 9, 5 11. Με `order=[2,1]` δίνει γραμμές 1 3, 5 7, 9 11. Έτσι φαίνεται ότι ο κανόνας ισχύει σε κάθε μορφή, όχι μόνο για `[2,3]`.

```
do i = 1, 2
  print '(3(i4))', (z(i,j), j=1,3)
end do
```

Τι να θυμάσαι για την Άσκηση 1: το `Y` ζητείται με προτεραιότητα γραμμών, άρα `order=[2,1]`. Το `Z` ζητείται με προτεραιότητα στηλών, άρα αρκεί η προεπιλογή `reshape(x,[2,3])`.

4.8 — Πρόσβαση και επεξεργασία στοιχείων

Χρησιμοποιούμε δείκτες για ένα στοιχείο και τμήματα με άνω-κάτω τελεία για συνεχόμενη περιοχή. Μπορούμε επίσης να κάνουμε πράξεις στοιχείο προς στοιχείο σε συστοιχίες ίδιας μορφής.

```
real :: a(5), b(5), c(5)
a = [1.0, -2.0, 3.0, 0.0, -8.0]
b = -a
c = a + b
print *, a(3) ! 3.0
print *, a(2:4) ! -2.0, 3.0, 0.0
```

* Τι σημαίνει `a(2:4)`: το `:` δηλώνει ένα συνεχόμενο τμήμα της συστοιχίας. Το `a(2:4)` δεν είναι ένα μόνο στοιχείο: επιλέγει τα στοιχεία από τον δείκτη 2 έως και τον δείκτη 4, δηλαδή `a(2)`, `a(3)` και `a(4)`. Εδώ οι τιμές τους είναι -2.0, 3.0, 0.0, και η `print` τις εμφανίζει με αυτή τη σειρά. Το άνω όριο συμπεριλαμβάνεται.

Για την πράξη `c=a+b`, κάθε θέση του `c` προκύπτει από τις αντίστοιχες θέσεις των `a` και `b`. Οι συστοιχίες πρέπει να έχουν την ίδια μορφή.

4.8.2 — Χρήσιμες εγγενείς συναρτήσεις συστοιχιών

Οι παρακάτω συναρτήσεις δέχονται μια συστοιχία ως όρισμα και επιστρέφουν ένα μόνο αποτέλεσμα. Με τη συστοιχία `a=[1.0,-2.0,3.0,0.0,-8.0]` έχουν τα εξής αποτελέσματα:

Συνάρτηση	Τι επιστρέφει	Αποτέλεσμα για <code>a</code>
<code>size(a)</code>	το πλήθος των στοιχείων	5
<code>sum(a)</code>	το άθροισμα όλων των στοιχείων	-6.0
<code>minval(a)</code>	τη μικρότερη τιμή	-8.0
<code>maxval(a)</code>	τη μεγαλύτερη τιμή	3.0

```
real :: a(5)
a = [1.0, -2.0, 3.0, 0.0, -8.0]
print *, size(a)
print *, sum(a)
print *, minval(a)
print *, maxval(a)
```

Τι εμφανίζει: 5, -6.0, -8.0, 3.0 (σε διαδοχικές γραμμές, με ελεύθερη μορφή). Το `size(a)` είναι ιδιαίτερα χρήσιμο μέσα σε DO, γιατί το πρόγραμμα μπορεί να χρησιμοποιεί το πραγματικό πλήθος στοιχείων χωρίς να γράφουμε εμείς χειροκίνητα τον αριθμό 5.

4.9 — Συνδυασμός arrays + DO

Η DO είναι ο ασφαλέστερος τρόπος όταν η τιμή κάθε στοιχείου εξαρτάται από τον δείκτη ή όταν θέλουμε να εκτυπώσουμε 2D πίνακα ανά γραμμή.

```
integer :: i, j
integer :: x(6), y(2,3)
x = [(2*i-1, i=1,6)]
y = reshape(x, [2,3], order=[2,1])
do i = 1, 2
  print '(3(i4))', (y(i,j), j=1,3)
end do
```

Τι εμφανίζει: δύο γραμμές: 1 3 5 και 7 9 11. Ο εξωτερικός βρόχος επιλέγει γραμμή `i`. Η υπονοούμενη επανάληψη στην εκτύπωση περνά από τις στήλες `j=1..3`.

4.10 — Συνδυασμός arrays + IF

Το IF χρησιμοποιείται όταν η επεξεργασία εξαρτάται από την τιμή ενός στοιχείου. Διαβάζουμε κάθε στοιχείο μέσα σε DO και αποφασίζουμε τι θα κάνουμε.

```
integer :: i, positive
real :: a(5)
a = [1.0, -2.0, 3.0, 0.0, -8.0]
positive = 0
do i = 1, size(a)
  if (a(i) > 0.0) positive = positive + 1
end do
print *, positive
```

Τι εμφανίζει: 2, επειδή θετικά είναι μόνο τα 1.0 και 3.0. Η `size(a)` αποφεύγει το να γράψουμε χειροκίνητα το 5.

4.11 — Μορφοποίηση και εκτύπωση πινάκων

Η μορφοποίηση κάνει τα δεδομένα αναγνώσιμα. Στο Εργαστήριο 3 ζητείται ρητά η εκτύπωση δεδομένων και αποτελεσμάτων σε κατάλληλο πίνακα. Παρακάτω είναι ένα πλήρες, αυτοτελές παράδειγμα για τα τρία ρευστά του Εργαστηρίου 3:

```
program thermal_power
implicit none
integer, parameter :: n = 3
integer :: i
real :: tout
real :: mdot(n), c(n), tin(n), pwr(n)

tout = 90.0
mdot = [1000.0, 800.0, 600.0]
c = [4.18, 2.00, 3.50]
tin = [25.0, 40.0, 60.0]
pwr = mdot*c*(tout-tin)

print '(a)', 'Fluid m_dot c Tin P'
do i = 1, n
write(*,'(i4,f10.2)') i, mdot(i), c(i), tin(i), pwr(i)
end do
write(*,'(a,f10.2)') 'P_total = ', sum(pwr)
end program thermal_power
```

Τι είναι το parameter: η δήλωση `integer, parameter :: n=3` δημιουργεί μια επώνυμη σταθερά. Το `n` κρατά πάντα την τιμή 3 και δεν μπορεί να αλλάξει μέσα στο πρόγραμμα. Θα μπορούσαμε να γράψουμε παντού το 3, αλλά το `n` δείχνει τι σημαίνει ο αριθμός: είναι το πλήθος των ρευστών. Έτσι χρησιμοποιούμε το ίδιο όνομα στις δηλώσεις `mdot(n)`, `c(n)`, κτλ. και στο `do i=1,n`. Αν συνότρεσα υπήρχαν 5 ρευστά, αλλάζουμε μόνο τη μία τιμή σε `n=5`.

Τι είναι το `a` στο format: το `a` δεν είναι μεταβλητή. Είναι ο περιγραφέας μορφής για κείμενο (αλφαριθμητικό). Δεν είναι απαραίτητο όταν χρησιμοποιούμε ελεύθερη εκτύπωση, π.χ. `print *`, `Fluid ... P`. Όταν όμως επιλέγουμε συγκεκριμένο format, πρέπει να υπάρχει ένας περιγραφέας για κάθε είδος δεδομένου. Στο `print (a), Fluid ... P`, το `(a)` λέει «τύπωσε το επόμενο στοιχείο ως κείμενο», άρα εμφανίζει την επικεφαλίδα. Στο `write(*,(a,f10.2)) P_total = , sum(pwr)`, το `a` τυπώνει πρώτα το κείμενο `P_total =` και το `f10.2` τυπώνει μετά τον αριθμό `sum(pwr)`. Χωρίς το `a`, η Fortran θα προσπαθούσε να εφαρμόσει το `f10.2` στο κείμενο, πράγμα που δεν ταιριάζει και οδηγεί σε σφάλμα.

Η `n=3` δηλώνει ότι υπάρχουν τρία ρευστά. Γι' αυτό το `do i=1,n` εκτελείται για `i=1, 2` και `3` και η `write` τυπώνει μία γραμμή για κάθε ρευστό. Το `i4` δίνει πεδίο πλάτους 4 για τον ακέραιο αριθμό του ρευστού.

Το `f10.2` σημαίνει ότι το `f10.2` επαναλαμβάνεται 4 φορές. Υπάρχουν ακριβώς τέσσερις πραγματικές τιμές μετά το `i`: `mdot(i)`, `c(i)`, `tin(i)` και `pwr(i)`. Κάθε `f10.2` δίνει πεδίο πλάτους 10 χαρακτήρων και εμφανίζει 2 δεκαδικά. Αν υπήρχε `3f10.2`, θα υπήρχαν format μόνο για τρεις πραγματικές τιμές, άρα δεν θα ταίριαζε σε αυτή τη γραμμή εκτύπωσης.

Στην τελευταία εντολή `write(*,(a,f10.2)) P_total = , sum(pwr)`, ο πρώτος αστερίσκος σημαίνει την τυπική έξοδο, δηλαδή την οθόνη. Το format `a` τυπώνει το κείμενο `P_total =`. Έπειτα το `f10.2` τυπώνει την τιμή του `sum(pwr)` σε πεδίο 10 χαρακτήρων με 2 δεκαδικά. Έτσι στην ίδια γραμμή εμφανίζονται η ετικέτα και η συνολική ισχύς. Για 2D πίνακα, συνήθως κάνουμε έναν DO για κάθε γραμμή και εκτυπώνουμε τα στοιχεία της γραμμής με κατάλληλο format.

4.12 — Από εκφώνηση σε πρόγραμμα

Για προβλήματα με πίνακες, ακολούθησε πάντα την ίδια σειρά:

- Δεδομένα: ποια μεγέθη είναι ένα ανά στοιχείο/ρευστό/μέτρηση; Αυτά γίνονται συστοιχίες.
- Διαστάσεις: πόσα στοιχεία υπάρχουν; γράψε τις εκτάσεις και έλεγξε το συνολικό μέγεθος.
- Αρχικοποίηση: χρησιμοποίησε constructor ή υπονοούμενη επανάληψη όταν οι τιμές έχουν μοτίβο.

- Υπολογισμός: επέλεξε πράξη ολόκληρης συστοιχίας ή DO όταν απαιτείται έλεγχος/μορφοποίηση.
- Έλεγχος: επιβεβαίωσε δείκτες, μονάδες και μορφή συστοιχιών.
- Παρουσίαση: τύπωσε επικεφαλίδες, μία γραμμή ανά δεδομένο και τελικό άθροισμα/αποτέλεσμα.

Παράδειγμα του Εργαστηρίου 3: για τρία ρευστά ορίζουμε `mdot(3)`, `c(3)`, `tin(3)`, `pwr(3)`, υπολογίζουμε `pwr = mdot*c*(tout-tin)` στοιχείο προς στοιχείο και βρίσκουμε `sum(pwr)`.

4.13 — Συχνά λάθη

Λάθος	Διόρθωση
Χρήση δείκτη 0 σε <code>a(5)</code>	Με τη συνηθισμένη δήλωση χρησιμοποίησε 1..5.
Σύγχυση έκτασης με δείκτη	Στο <code>a(5)</code> , το 5 είναι έκταση στη δήλωση· στο <code>a(i)</code> , το <code>i</code> είναι δείκτης.
Λάθος πλήθος τιμών στον constructor	Ο αριθμός στοιχείων πρέπει να συμφωνεί με το μέγεθος.
Πράξη <code>arrays</code> διαφορετικής μορφής	Για <code>a+b</code> απαιτείται ίδια μορφή.
Αναμονή <code>row-major</code> από το <code>RESHAPE</code>	Η προεπιλογή είναι κατά πρώτη διάσταση· για κατά γραμμές χρησιμοποίησε <code>order=[2,1]</code> .
Εκτύπωση 2D χωρίς έλεγχο γραμμών	Χρησιμοποίησε <code>DO</code> και <code>format</code> ανά γραμμή.
Παράλειψη <code>implicit none</code>	Δήλωνε πάντα ρητά όλες τις μεταβλητές.

4.14 — Ασκήσεις εφαρμογής

1. Δημιούργησε με υπονοούμενη επανάληψη τον μονοδιάστατο πίνακα $X = [1, 3, 5, 7, 9, 11]$. Με `RESHAPE` δημιούργησε $Y(2,3)$ με προτεραιότητα γραμμών και $Z(2,3)$ με προτεραιότητα στηλών. Εκτύπωσε X , Y και Z ανά γραμμή, με μορφοποίηση της επιλογής σου.
2. Δημιούργησε διάνυσμα 100 ακεραίων. Στη θέση i ($i=1,...,100$) δώσε την τιμή i^2+3i+1 . Υπολόγισε τον μέσο όρο των στοιχείων του.
3. Ο χρήστης δίνει N . Δημιούργησε διάνυσμα με τιμή $\sin(\pi j/N)$ στη θέση j . Βρες μέγιστο, ελάχιστο και πόσα στοιχεία έχουν απόλυτη τιμή μεγαλύτερη από 0.4.
4. Για 3 ρευστά με δεδομένα $\dot{m}=[1000,800,600]$ kg/h, $c=[4.18,2.00,3.50]$ kJ/(kg·°C), $T_{in}=[25,40,60]$ °C και $T_{out}=90$ °C, υπολόγισε $P=\dot{m}c(T_{out}-T_{in})$ για κάθε ρευστό και τη συνολική ισχύ. Δώσε τα δεδομένα με constructor και τύπωσε πίνακα αποτελεσμάτων.
5. Δημιούργησε δύο πραγματικούς πίνακες A , B διαστάσεων 20×30 . Δώσε $A(i,j)=(i+j)/3$ και $B(i,j)=2i-j/3$. Υπολόγισε τον ανάστροφο του B , το γινόμενο $A \cdot B^T$ και το άθροισμα της κύριας διαγωνίου του γινομένου.

4.15 — Checklist Κεφαλαίου 4

- Μπορώ να εξηγήσω τι σημαίνουν τάξη, μέγεθος, μορφή, έκταση και δείκτης.
- Μπορώ να δηλώσω σωστά 1D και 2D συστοιχία με `implicit none`.
- Γνωρίζω ότι ο συνηθισμένος πρώτος δείκτης είναι 1 και ελέγχω τα όρια.
- Μπορώ να χρησιμοποιήσω constructor και υπονοούμενη επανάληψη.
- Μπορώ να προβλέψω τη διαφορά ανάμεσα σε `reshape(x,[2,3])` και `reshape(x,[2,3],order=[2,1])`.
- Μπορώ να προσπελάσω στοιχείο και τμήμα συστοιχίας.
- Ξέρω ότι οι πράξεις ολόκληρων συστοιχιών απαιτούν συμβατή μορφή.
- Μπορώ να συνδυάζω `arrays` με `DO` και `IF`, να τα εκτυπώνω με `FORMAT` και, πριν την παράδοση, να ελέγχω διαστάσεις, μονάδες, δείκτες και αναγνωσιμότητα εξόδου.